FlashSampling: Fast and Memory-Efficient Exact Sampling

FlashSampling Authors

FlashSampling Project

March 6, 2026

Abstract

Sampling from categorical distributions is a ubiquitous inner-loop primitive in modern deep learning systems, appearing in LLM inference, mixture-of-experts routing, negative sampling, sequence generation, and reinforcement learning. At large category counts V, sampling is frequently dominated by memory movement: conventional softmax-based pipelines materialize $[B, V]$ logits, perform multiple passes, and, in tensor-parallel LM heads, may communicate $O(V)$ values per row across devices. We present **FlashSampling**, a unified algorithmic and kernel primitive for *exact* categorical sampling that is fast, memory efficient, and communication efficient.

FlashSampling uses a fused tiled first-stage kernel, followed by a lightweight reduction over tile winners: logits are computed tile-by-tile, perturbed with Gumbel noise, and reduced on-chip, so the logits tensor itself is never written to HBM. We prove exactness of the tiled fused design by argmax decomposition over tiles, and use Gumbel max-stability to derive grouped, streaming, and distributed variants. A systems analysis shows that raw logits-byte savings alone are too small to explain the observed gains in decode; the main benefit is the elimination of extra post-matmul kernels and their launch/synchronization overhead.

In kernel-level benchmarks across four NVIDIA GPUs (H100, H200, B200, B300), FlashSampling achieves up to 1.5× speedup over PyTorch compiled sampling and up to 2.3× over FlashInfer baselines, with the gains concentrated in the memory-bound decode regime (batch size ≤ 64). In end-to-end benchmarks on vLLM, FlashSampling reduces time per output token (TPOT) by up to 19% on Qwen3-1.7B and 3–7% on Qwen3-8B, with no degradation in generation quality when verified on the GSM8K dataset.

`Project Page:` https://github.com/FlashSampling/FlashSampling

1 Introduction

Sampling from a categorical distribution with probabilities $p_i \propto \exp(\ell_i)$ is arguably the simplest stochastic primitive, yet it is a persistent bottleneck in large-scale systems. In modern LLMs and MoE models, the number of categories V or experts can be in the tens to hundreds of thousands. A typical pipeline computes logits $\boldsymbol{\ell} \in \mathbb{R}^V$ and then samples an index $i \sim \mathrm{Cat}(\mathrm{softmax}(\boldsymbol{\ell}))$. When implemented naively, this requires writing $\boldsymbol{\ell}$ to high-bandwidth memory (HBM), reading it back for sampling, and performing multiple passes for normalization, prefix sums, or rejection sampling. These operations are often bandwidth-bound and incur significant kernel-launch overhead.

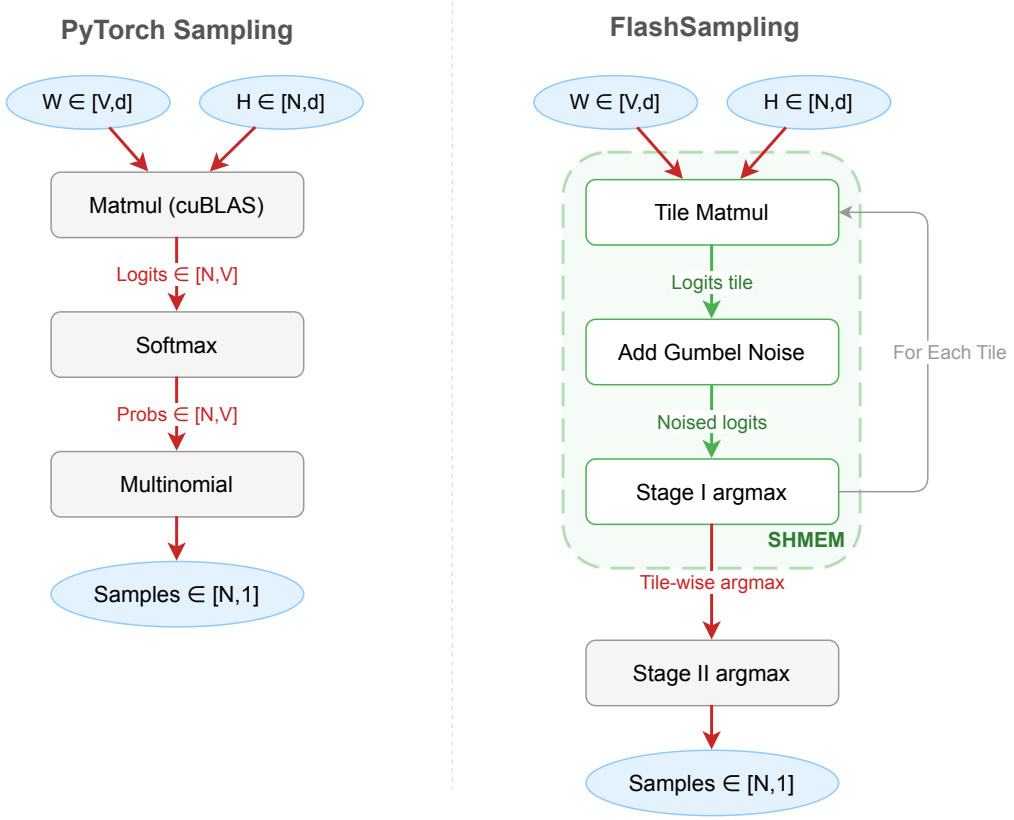

Figure 1 Baseline pipeline (left) materializes the full $[B, V]$ logits tensor in HBM between the matmul and sampler. FlashSampling (right) fuses sampling into the matmul epilogue: logits are computed tile-by-tile in on-chip memory, perturbed with Gumbel noise, and reduced to per-tile winners without ever writing logits to HBM.

In autoregressive decoding, sampling is invoked at every step with small batch sizes (often $B \leq 64$), where the LM-head projection and the sampler are typically *memory-bound*: the dominant cost lies in streaming tensors from/to HBM. Therefore, repeatedly reading and writing the short-lived $[B, V]$ logits *directly* adds to the total decode runtime.

Our contributions are:

1. **The FlashSampling algorithm**: a fused tiled first-stage kernel, followed by a lightweight reduction, that combines the LM-head matmul with Gumbel-Max sampling on-chip. Logits are computed tile-by-tile in SRAM/registers, perturbed with Gumbel noise, and reduced to per-tile winners without ever materializing the $[B, V]$ logits tensor in HBM.

2. **Theory of exactness and correctness**: Exactness has two ingredients. For the fused tiled kernel, exactness follows from argmax decomposition over partitions: reducing tile-local maxima and then reducing across tiles returns the same global winner as a full argmax. For

grouped, streaming, and distributed variants, Gumbel max-stability under grouping yields an exact hierarchical factorization. These facts also support masking and other constrained variants.

3. **Systems analysis**: an IO-complexity cost model and kernel profiles identifying the dominant speedup sources in actual practice. A key insight is that the fused sampler adds only a small roughly constant overhead to the logits computation, whereas unfused pipelines pay extra kernel, synchronization, and memory-system costs.

2 Background

Notation. Let $[V] := \{1, \ldots, V\}$ and let $\boldsymbol{\ell} \in \mathbb{R}^V$ be logits. The categorical distribution is $p(i) = \exp(\ell_i)/\sum_{j=1}^{V} \exp(\ell_j)$. We denote i.i.d. standard Gumbel variables by $g_i \sim \mathrm{Gumbel}(0, 1)$. Because the Gumbel law is continuous, ties occur with probability zero; argmax therefore returns a unique index almost surely.

2.1 Why sampling is expensive at scale

A typical GPU sampling stack for logits $\boldsymbol{\ell}$ might compute probabilities $\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{\ell})$, generate a uniform random number u, build a prefix sum of $\boldsymbol{p}$, and find the smallest index with cumulative mass exceeding u. This implies at least two passes over $\boldsymbol{\ell}$ (to compute max and sum-exp), plus writing/reading intermediate buffers. The cost of these operations is dominated by HBM bandwidth, not arithmetic. When logits are produced by a GEMM, e.g. $\boldsymbol{\ell} = \boldsymbol{W}\boldsymbol{h}$ or $\boldsymbol{H}\boldsymbol{W}^\top$, the default decomposition is:

$$\text{GEMM(produce logits)} \;\rightarrow\; \text{write logits to HBM} \;\rightarrow\; \text{read logits for sampling.}$$

This forces a full materialization of an ephemeral logits tensor. Algorithm 1 summarizes the standard multinomial sampling pipeline that FlashSampling replaces.

Algorithm 1 Standard multinomial sampling pipeline

Require: Hidden state $\boldsymbol{h} \in \mathbb{R}^D$, LM-head weights $\boldsymbol{W} \in \mathbb{R}^{V \times D}$, temperature $\tau > 0$
Ensure: Sampled index $i^\star \in \{1, \ldots, V\}$

```
ℓ ← Wh                                  ▷ GEMM: compute logits (write ℓ to HBM)
ℓ ← ℓ/τ                                 ▷ temperature scaling (read/write ℓ)
m ← max_i ℓ_i                           ▷ pass 1: numerical stability (read ℓ)
Z ← Σ_{i=1}^{V} exp(ℓ_i − m)            ▷ pass 2: normalizer (read ℓ)
p_i ← exp(ℓ_i − m)/Z for all i          ▷ write p to HBM
c_i ← Σ_{j=1}^{i} p_j for all i         ▷ pass 3: prefix sum (read p, write c)
Draw u ~ Unif(0, 1)
i* ← min{i : c_i ≥ u}                   ▷ pass 4: search (read c)
return i*
```

Decode regime. In autoregressive decoding, B is typically small ($B \leq 64$). The LM-head projection is memory-bound because it streams the large $[V, D]$ weight matrix from HBM at each step, and performs a thin matrix multiplication with the hidden states. Materializing $[B, V]$ logits and reading them back several times for sampling adds multiple, avoidable HBM round-trips. FlashSampling targets this regime explicitly via fusion (Section 3.4).

2.2 GPU Memory Hierarchy

Table 1 summarizes the GPU memory hierarchy. On-chip memory (registers, SRAM) is orders of magnitude faster than HBM but orders of magnitude smaller. FlashSampling exploits this gap by keeping logits in registers/SRAM and never writing them to HBM.

Table 1 GPU memory hierarchy (H100 SXM) (NVIDIA, 2022, 2024).

Level	Capacity	Bandwidth
Registers/SRAM	256 KB / SM	>100 TB/s
L2 cache	50 MB	~ 12 TB/s
HBM3	80 GB	3.35 TB/s

2.3 The Gumbel-Max trick

The classical Gumbel-Max trick states that sampling from a categorical distribution can be performed by adding i.i.d. Gumbel noise and taking an argmax:

Theorem 2.1 (Gumbel-Max)**.** Let $\{g_i\}_{i=1}^{V}$ be i.i.d. Gumbel$(0, 1)$. Then

$$i^\star = \operatorname*{argmax}_{i \in [V]} (\ell_i + g_i) \quad \Longrightarrow \quad \mathbb{P}(i^\star = i) = \frac{e^{\ell_i}}{\sum_{j=1}^{V} e^{\ell_j}}.$$

A standard proof uses the fact that $\max_i(\ell_i + g_i)$ has a Gumbel distribution with location $\log \sum_j e^{\ell_j}$ and that the argmax probabilities match the softmax.

3 FlashSampling

We now describe FlashSampling. We begin with the core exact sampling algorithm (standalone logits), then show how to fuse it with GEMM, and finally discuss extensions and practical concerns.

3.1 Exact sampling via streaming Gumbel-Max

Given logits $\boldsymbol{\ell} \in \mathbb{R}^V$, we wish to sample $i \sim \mathrm{Cat}(\mathrm{softmax}(\boldsymbol{\ell}))$.

Algorithm (one-pass). Generate i.i.d. $g_i \sim$ Gumbel$(0, 1)$, compute $s_i = \ell_i + g_i$, and return $i^\star = \mathrm{argmax}_i\, s_i$. This can be implemented in a single streaming pass that maintains the running maximum and index. No normalization is required (see Algorithm A.1 in the Appendix for detailed pseudocode).

Systems implication. Sampling reduces to a *single pass* over logits to compute the maximum of perturbed values and record the argmax. This naturally fits GPU reductions and eliminates prefix sums.

GPU parallelization. For batch size B, logits form a matrix $\boldsymbol{L} \in \mathbb{R}^{B \times V}$. We assign a threadblock to each row (or to a group of rows), generate Gumbels per element, compute partial maxima over tiles of V, and reduce to the row-wise maximum with argmax.

This mirrors common reductions, but FlashSampling emphasizes *blockwise streaming* when V is large:

- Load a tile of logits into registers (or shared memory if beneficial),
- Generate matching Gumbels on-the-fly (no storing),
- Compute perturbed scores and maintain per-thread maxima,
- Reduce maxima across threads to a block-level maximum and index,
- Optionally perform a second-level reduction across blocks if V is too large for one block.

3.2 Group-Gumbel-Max: hierarchical exact sampling

A key algorithmic contribution of FlashSampling is the *Group-Gumbel-Max* factorization, which enables exact sampling without materializing full logits or probabilities. Partition $[V]$ into m disjoint groups $\{\mathcal{G}_k\}_{k=0}^{m-1}$, each of size g (so $V = mg$). Define group log-masses

$$L_k \;=\; \log \sum_{i \in \mathcal{G}_k} \exp(\ell_i) \;=\; \mathrm{logsumexp}(\boldsymbol{\ell}_{\mathcal{G}_k}).$$

Then the categorical distribution factorizes as

$$\underbrace{\mathbb{P}(K = k)}_{\text{pick group}} \propto \exp(L_k), \qquad \underbrace{\mathbb{P}(I = i \mid K = k)}_{\text{pick within group}} \propto \exp(\ell_i) \;\text{ for } i \in \mathcal{G}_k.$$

Group-Gumbel-Max samples K with Gumbel-Max on $\{L_k\}$ and samples I with Gumbel-Max on the logits in the chosen group, yielding an exact sample from the full V-way categorical while (crucially) permitting streaming and distributed implementations that never materialize the full V logits or probabilities.

The key insight enabling this is that Gumbel perturbations are *max-stable*: the maximum of Gumbel-perturbed logits within a group is itself Gumbel-distributed with location $L_k = \mathrm{logsumexp}(\boldsymbol{\ell}_{\mathcal{G}_k})$ (Lemma 4.1). Equivalently, one can sample hierarchically: first sample a group from logits $\{L_k\}$, then sample within the selected group from the restricted logits. Either view is exact, and the factorization naturally supports parallel, sequential/online, and distributed execution.

Parallel FlashSampling. Assume logits arise from a linear projection $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$, where $\boldsymbol{W} \in \mathbb{R}^{V \times D}$ and $\boldsymbol{x} \in \mathbb{R}^D$ (typical LM head / routing). Let $\boldsymbol{W}_{\mathcal{G}_k} \in \mathbb{R}^{g \times D}$ be the block of rows indexed by group $\mathcal{G}_k$, so $\boldsymbol{y}_k = \boldsymbol{W}_{\mathcal{G}_k}\boldsymbol{x} \in \mathbb{R}^g$ are the group logits. Parallel FlashSampling computes all groups (e.g., across threadblocks) and reduces over groups: each group draws a local Gumbel-Max sample z_k and computes the group log-mass $L_k = \mathrm{logsumexp}(\boldsymbol{y}_k)$. A second-level Gumbel-Max over the m group log-masses selects the winning group $k^\star$; the global sample is $z_{k^\star}$ mapped to its global vocabulary index (Algorithm A.2).

Sequential/online FlashSampling. When memory is the primary constraint (e.g., very large V or long sequences), FlashSampling provides a sequential variant that streams groups one at a time, maintaining only a running log-mass L_{run} and a running sample z. After initializing from the first group, each subsequent group k computes its log-mass L_k and combines it with L_{run} via logsumexp. The running sample is replaced by the new group's local Gumbel-Max winner with probability $\exp(L_k - L_{\text{new}})$, where $L_{\text{new}} = \log(e^{L_{\text{run}}} + e^{L_k})$. This requires only $O(g)$ working memory regardless of V (Algorithm A.3).

Exactness. Group-Gumbel-Max is exact because it samples from the correct hierarchical factorization of the categorical distribution (group selection by $\exp(L_k)$, then within-group selection by $\exp(\ell_i)$ restricted to the group). Section 4.2 provides a short proof.

Why it helps. On GPU, grouping enables:

- **less materialization:** avoid storing full logits/probabilities; store only group statistics (parallel) or keep only one group in flight (sequential),
- **structured parallelism:** map groups to threadblocks/warps with contiguous memory access,
- **distributed reduction:** treat tensor-parallel shards as groups and communicate only (z_k, L_k), not logits.

3.3 Distributed FlashSampling for tensor-parallel vocabularies

In tensor-parallel LM heads, the vocabulary dimension is sharded across n GPUs. Naively, each GPU computes local logits and then an all-gather concatenates the full V logits before sampling, incurring $O(V)$ communication per row. FlashSampling reduces this to $O(n)$ communicated values per row, independent of V: each rank returns (i) a local sample z_k from its shard and (ii) the shard log-mass $L_k = \text{logsumexp}(\boldsymbol{y}^{(k)})$. A final Group-Gumbel-Max over the n shards chooses which shard provides the global sample.

Status. The distributed algorithm is fully specified (Algorithm A.4) and its correctness follows from the same Group-Gumbel-Max exactness guarantees (Theorem 4.3). However, we have not yet implemented or benchmarked it; the empirical evaluation in Section 6 focuses on single-GPU FlashSampling. We leave the distributed implementation and evaluation as future work.

3.4 FlashSampling for LM-head sampling

GEMM kernels already compute partial dot-products in registers/shared memory and then write final outputs in an *epilogue*. If we incorporate Gumbel perturbation and a tile-local argmax into that epilogue, logits need not be stored; a lightweight second-stage reduction over tile winners then completes the sample. FlashSampling provides the algorithmic structure (grouped maxima and streaming) to make this fusion practical, improving occupancy and extending naturally to distributed tensor-parallel sampling.

We now consider the common case where logits are produced by GEMM. Let

$$\boldsymbol{Y} = \boldsymbol{H}\boldsymbol{W}^\top \in \mathbb{R}^{B \times V},$$

Algorithm 2 FlashSampling fused matmul-sample (two-stage): tile-local winners then reduction

Require: Hidden states $\boldsymbol{H} \in \mathbb{R}^{B\times D}$, LM-head weights $\boldsymbol{W} \in \mathbb{R}^{V\times D}$, temperature $\tau > 0$, RNG seed
Ensure: Samples $\boldsymbol{i}^\star \in \{1, \ldots, V\}^B$

Stage 1 (kernel): for each vocabulary tile t (and batch tile) in parallel

Let tile start index be v_t and tile vocab indices be $\mathcal{T}_t = \{v_t + 1, \ldots, \min(v_t + T, V)\}$
Compute logits tile $\boldsymbol{Y}^{(t)} \leftarrow \boldsymbol{H}\,(\boldsymbol{W}_{\mathcal{T}_t})^\top$ ▷ accumulate in SRAM/registers
$\boldsymbol{S}^{(t)} \leftarrow \boldsymbol{Y}^{(t)}/\tau + \boldsymbol{G}^{(t)}$ ▷ $\boldsymbol{G}$ i.i.d. Gumbel, generated in-kernel
$(\boldsymbol{m}^{(t)}, \boldsymbol{j}^{(t)}) \leftarrow \mathrm{argmax}_{\text{vocab in tile}}\, \boldsymbol{S}^{(t)}$ ▷ per row, $j^{(t)} \in \{1, \ldots, |\mathcal{T}_t|\}$
$\mathbf{idx}^{(t)} \leftarrow v_t + \boldsymbol{j}^{(t)}$ ▷ convert tile-local indices to global vocab indices
Write $(\boldsymbol{m}^{(t)}, \mathbf{idx}^{(t)})$ to GMEM ▷ $O(B)$ per tile

Stage 2 (reduction): for each row b

$t^\star \leftarrow \mathrm{argmax}_t\, m_b^{(t)}$ ▷ which tile won?
$i_b^\star \leftarrow \mathrm{idx}_b^{(t^\star)}$ ▷ return that tile's global index
return $\boldsymbol{i}^\star$

where $\boldsymbol{H} \in \mathbb{R}^{B\times D}$ are hidden states and $\boldsymbol{W} \in \mathbb{R}^{V\times D}$ are LM-head weights. We wish to sample one index per row from $\mathrm{softmax}(\boldsymbol{Y}_{b,:})$.

Goal: avoid materializing $\boldsymbol{Y}$. FlashSampling performs sampling *inside* the matmul kernel and writes only $O(B)$ outputs (or, in a tiled design, $O(B \cdot \#V\text{-tiles})$ candidates), never materializing $\boldsymbol{Y}$ in GMEM:

- Stage 1 (fused kernel): compute logits tile-by-tile on-chip, apply temperature, add Gumbel noise, and compute the tile-local argmax along the vocabulary-tile dimension; write only the winning value and its vocabulary index per tile.
- Stage 2 (reduction): reduce over the per-tile winners to select the global winner per row, yielding the sampled token indices.

Implementation note. In our Triton reference implementation, the fused first stage is written in Triton and exposes an interface of the form `fused_mm_sample_triton(weights, hidden_states, num_samples, temperature, seed)`. The analysis in this paper focuses on the single-sample case (`num_samples=1`); multiple i.i.d. samples are obtained by repeating the same construction with independent RNG streams. The kernel uses a counter-based Philox RNG to generate uniforms in $(0, 1)$ and transforms them into Gumbels in-kernel. More generally, a fused epilogue can include:

- optional bias addition (e.g. vocabulary bias, ALiBi-like offsets),
- masking (e.g. to forbid tokens),
- temperature scaling $\ell \leftarrow \ell/\tau$,
- Gumbel generation and max tracking.

Tensor-parallel fusion. When the vocabulary is sharded across ranks, each rank can run the fused kernel on its local shard (producing local candidates and/or $(\widetilde{z}_k, L_k)$), and a lightweight cross-rank reduction selects the global sample without any $O(V)$ all-gather of logits.

3.5 Masking, forbidden indices, and structured constraints

In many applications, some categories are invalid (e.g. banned tokens, padding, or constraints from a decoding grammar). FlashSampling supports *hard* masks by excluding forbidden indices from both the local and global argmax. Mathematically, this is equivalent to setting their logits to $-\infty$ before perturbation:

$$s_i = \begin{cases} \ell_i + g_i, & \text{if allowed,} \\ -\infty, & \text{if forbidden.} \end{cases}$$

This preserves exactness over the restricted support, provided each row has at least one allowed index. A finite negative sentinel is exact only if the implementation separately enforces that masked entries can never be selected; otherwise it introduces a small approximation.

3.6 Top-k and nucleus variants

Although the core paper focuses on exact sampling over all V, practical decoding often uses truncated supports.

- **Exact top-k on a given support:** if an exact top-k set has already been identified, then Gumbel-Max within that set samples exactly from the top-k-truncated distribution.
- **Nucleus (top-p):** identifying the exact nucleus set requires additional cumulative-mass computation, so we treat it as an extension rather than part of the main exactness theorem.
- **Approximate variants:** coarse prefiltering (e.g. group-wise screening) can reduce the candidate set further, but then the resulting distribution is approximate.

The core contribution of this paper is exact sampling from the full categorical without materializing logits.

4 Analysis of FlashSampling

4.1 A unifying view: max-stability of grouped Gumbel perturbations

Group-Gumbel-Max and FlashSampling both rely on the fact that *max* decomposes over partitions (groups/tiles), and that Gumbel noise is *max-stable*. This yields a convenient bridge between (i) sampling via group log-masses and (ii) sampling via tile-local winners.

Lemma 4.1 (Gumbel max-stability under grouping)**.** Let $\{g_i\}_{i=1}^{V}$ be i.i.d. Gumbel$(0,1)$ and let $\{\mathcal{G}_k\}_{k=0}^{m-1}$ be a partition of $[V]$. Define

$$M_k \;=\; \max_{i\in\mathcal{G}_k}(\ell_i + g_i), \qquad I_k \;=\; \operatorname*{argmax}_{i\in\mathcal{G}_k}(\ell_i + g_i), \qquad L_k \;=\; \log \sum_{i\in\mathcal{G}_k} e^{\ell_i}.$$

Then (i) $M_k \sim \text{Gumbel}(L_k, 1)$, (ii) $\{M_k\}$ are independent across disjoint groups, and (iii) $\mathbb{P}(I_k = i) = e^{\ell_i} / \sum_{j\in\mathcal{G}_k} e^{\ell_j}$ for $i \in \mathcal{G}_k$.

Proof. For any real t,

$$\mathbb{P}(M_k \le t) = \prod_{i\in\mathcal{G}_k} \mathbb{P}(g_i \le t - \ell_i) = \prod_{i\in\mathcal{G}_k} \exp\big(-e^{-(t-\ell_i)}\big) = \exp\Big(-e^{-(t-L_k)}\Big),$$

which is the CDF of Gumbel$(L_k, 1)$. Independence follows because the groups are disjoint and the underlying $\{g_i\}$ are independent. The within-group argmax probabilities are exactly the Gumbel-Max trick applied to the restricted logits $\boldsymbol{\ell}_{\mathcal{G}_k}$. □

Consequence. Selecting the winning group by $\operatorname{argmax}_k M_k$ is equivalent in distribution to applying Gumbel-Max to group logits $\{L_k\}$, which justifies Group-Gumbel-Max group selection. For the fused two-stage kernel in Algorithm 2, however, exactness does *not* rely on max-stability: once the perturbed scores $x_i = \ell_i/\tau + g_i$ have been formed, exactness follows simply from $\max_i x_i = \max_t \max_{i \in \mathcal{T}_t} x_i$.

4.2 Exactness of Group-Gumbel-Max

The correctness of FlashSampling rests on two facts: (i) Group factorization of the categorical distribution is exact, and (ii) Gumbel-Max yields exact categorical samples from logits.

Lemma 4.2 (Exact group factorization)**.** Let $[V]$ be partitioned into groups $\{\mathcal{G}_k\}_{k=0}^{m-1}$. Define $L_k = \log \sum_{i \in \mathcal{G}_k} \exp(\ell_i)$. If we sample $K \sim \mathrm{Cat}(\mathrm{softmax}(\boldsymbol{L}))$ and then sample $I \mid (K = k) \sim \mathrm{Cat}(\mathrm{softmax}(\boldsymbol{\ell}_{\mathcal{G}_k}))$, the marginal distribution of I equals $\mathrm{Cat}(\mathrm{softmax}(\boldsymbol{\ell}))$.

Proof. For any $i \in \mathcal{G}_k$,

$$\mathbb{P}(I = i) \;=\; \mathbb{P}(K = k)\,\mathbb{P}(I = i \mid K = k) \;=\; \frac{e^{L_k}}{\sum_s e^{L_s}} \cdot \frac{e^{\ell_i}}{\sum_{j \in \mathcal{G}_k} e^{\ell_j}} \;=\; \frac{e^{\ell_i}}{\sum_{j=1}^{V} e^{\ell_j}}.$$

□

Theorem 4.3 (Group-Gumbel-Max exactness)**.** Algorithms A.2-A.3 return an exact sample from $\mathrm{Cat}(\mathrm{softmax}(\boldsymbol{\ell}))$.

Proof. By the lemma, it suffices to sample K from logits $\{L_k\}$ and I within the chosen group from logits $\boldsymbol{\ell}_{\mathcal{G}_K}$. Each step is exact via the Gumbel-Max trick. □

4.3 Exactness of tile-wise FlashSampling reduction

FlashSampling relies on a standard decomposition: the global maximum equals the maximum of tile-local maxima.

Lemma 4.4 (Max over tiles)**.** Let $\{x_i\}_{i=1}^{V}$ be real numbers and let $\{\mathcal{T}_s\}_{s=0}^{n_{\text{tile}}-1}$ be a partition of $[V]$ into tiles. For each tile, define

$$m_s = \max_{i \in \mathcal{T}_s} x_i, \qquad \widehat{\imath}_s \in \operatorname*{argmax}_{i \in \mathcal{T}_s} x_i,$$

where $\widehat{\imath}_s$ is a global index in $\mathcal{T}_s$. Then

$$\max_{i \in [V]} x_i = \max_s m_s.$$

Moreover, for any $s^\star \in \operatorname{argmax}_s m_s$, the chosen index $\widehat{\imath}_{s^\star}$ is a global maximizer. Conversely, every global maximizer lies in some tile $s^\star \in \operatorname{argmax}_s m_s$.

Proof. The identity for the maximum value is immediate:

$$\max_{i\in[V]} x_i = \max_s \max_{i\in\mathcal{T}_s} x_i = \max_s m_s.$$

If $s^\star \in \operatorname{argmax}_s m_s$, then $x_{\widehat{\imath}_{s^\star}} = m_{s^\star} = \max_i x_i$, so $\widehat{\imath}_{s^\star}$ is a global maximizer. Conversely, if $i^\star$ is any global maximizer, then its tile $s^\star$ satisfies $m_{s^\star} = x_{i^\star} = \max_i x_i$, hence $s^\star \in \operatorname{argmax}_s m_s$. □

Applying this lemma to $x_i = \ell_i/\tau + g_i$ justifies the two-stage FlashSampling design in Algorithm 2. Because the Gumbel variables are continuous, the global maximizer is unique almost surely, so the two-stage reduction returns exactly the same index as a full row-wise argmax with probability one.

4.4 Cost model: bandwidth vs. compute

We outline a simple model to reason about speedups.

Standalone sampling from materialized logits. Let B be batch size and V be categories. A naive pipeline reads logits (BV floats) and performs softmax and prefix-sum-like sampling. This typically requires multiple passes:

- pass 1: read logits to compute max,
- pass 2: read logits to compute sum-exp,
- pass 3: sample via prefix sum / search.

Even if some operations fuse, a substantial fraction of BV values is read multiple times.

Streaming Gumbel-Max. FlashSampling reads logits once and performs a single reduction. Memory traffic is roughly one read of logits (plus RNG state), and the output is only B indices.

Fused GEMM + sampling. If logits are produced by GEMM, the dominant cost is usually GEMM compute. However, for large V, writing $\boldsymbol{Y} \in \mathbb{R}^{B\times V}$ to HBM and reading it back can be comparable to GEMM time. Fusion eliminates this write/read pair and replaces it with:

- on-chip (SRAM/registers) perturbation and tile-local reduction,
- a small second-stage reduction over tile winners,
- writing only sampled indices (and optionally additional statistics).

Thus, fusion is most beneficial when:

- $[B, V]$ is large (logits tensor is big),
- sampling is invoked frequently (inner-loop),
- the system is bandwidth-limited or launch-overhead dominated.

Incremental traffic saved by fusion. Relative to a fused kernel, a baseline that materializes logits incurs at least one write and one read of the $[B, V]$ logits tensor. In BF16 this is an additional $4BV$ bytes; a multi-pass softmax sampler adds a larger constant multiple of BV. The LM-head weight read is $2VD$ bytes, so the minimum extra-traffic ratio is

$$\frac{4BV}{2VD} = \frac{2B}{D}.$$

For the small config ($D = 4096$), this ratio is 0.049% at $B = 1$, 3.125% at $B = 64$, and 6.25% at $B = 128$. Thus raw logits-byte savings alone are too small to explain the largest measured speedups. The main gains come from eliminating extra post-matmul kernels, global-memory round-trips through those kernels, and their launch/synchronization overhead.

4.5 Optional: returning log-normalizers or max values

Some applications need $\log Z = \log \sum_j e^{\ell_j}$ (e.g. computing log-probabilities). The Gumbel-Max argmax does not directly provide $\log Z$, but the same streaming pass can compute both:

- compute $m = \max_i \ell_i$,
- compute $\sum_i \exp(\ell_i - m)$,
- output $\log Z = m + \log \sum_i \exp(\ell_i - m)$,

at the cost of an extra reduction pass or additional work in the same pass (with care). In fused GEMM settings, returning $\log Z$ requires accumulating exp terms, which may reduce performance; FlashSampling treats this as an optional mode.

5 Kernel Design: Practical FlashSampling

This section sketches an implementable GPU kernel design for FlashSampling.

5.1 Standalone kernel: logits → sample

Threadblock mapping. For each row (batch element) b, assign a threadblock. Each thread processes a strided subset of indices i. This is a standard parallel reduction.

Per-thread work. Each thread:

- loads ℓ_i,
- generates g_i,
- computes $s_i = \ell_i + g_i$,
- tracks $(m, i^\star)$ in registers.

Reduction. Use warp-level primitives to reduce maxima and argmax indices. For large V, use multi-stage reduction: within warps, then across warps using shared memory.

5.2 Fused GEMM epilogue: matmul → sample

GEMM structure. Assume a blocked GEMM producing tiles $\boldsymbol{Y}_{b,v:v+t}$ in registers.

Epilogue fusion. Instead of writing $\boldsymbol{Y}$, the fused kernel:

- applies optional transforms (bias, temperature, mask),
- generates Gumbel noise for each output element,
- updates per-row maxima and argmax indices,
- discards intermediate logits.

Cross-tile reduction. If a row spans multiple tiles computed by different threadblocks, we need to combine their maxima. Two common approaches:

- **Two-stage approach (FlashSampling-style):** each tile writes its per-row candidate (max value + argmax) to a small buffer; a cheap reduction (e.g., a PyTorch argmax or a second kernel) reduces across tiles. This still avoids materializing logits and is cheap because the buffer is size $\approx [B, \#V\text{-tiles}]$.
- **Persistent kernel / cooperative groups:** use a single kernel that processes all tiles for a row and reduces internally. This can be faster but more complex.

Our reference implementation uses the two-stage design for simplicity and strong performance in the decode regime.

5.3 RNG state management

Determinism. For reproducibility, RNG streams should be indexed by (b, i) or by a counter derived from the global position in $\boldsymbol{Y}$. Counter-based RNGs (e.g. Philox) are well-suited: each element's random number is a deterministic function of (key, counter), avoiding per-thread stateful updates.

Parallel safety. Each thread must generate independent random values for its assigned elements. Counter-based RNG naturally supports this by using distinct counters for each element.

5.4 Precision and data types

Logits dtype. Logits may be in FP16/BF16 in inference, but accumulation in GEMM often uses FP32. In the two-stage design, FlashSampling should:

- compute perturbed scores in FP32 for stability,
- write one FP32 winner value and one integer index per row/tile for the second-stage reduction,
- write only the final index if a persistent single-kernel reduction is used instead.

Gumbel noise dtype. Compute g in FP32 to avoid inaccuracies in logs. The overhead is typically minor compared to GEMM compute.

6 Experiments

We evaluate FlashSampling at two levels: (i) kernel-level microbenchmarks that isolate the fused matmul and sample operation across four GPU architectures, and (ii) end-to-end vLLM integration that measures the impact on autoregressive decode latency for four LLMs. All benchmarks use the open-source FlashSampling Triton implementation (Ruiz, 2026).

6.1 Setup

Hardware. Kernel microbenchmarks are run on four NVIDIA GPUs spanning two architecture generations. Table 2 summarizes their specifications. All GPUs are provisioned via Modal cloud.

Software. PyTorch 2.10.0, CUDA 13.0, Triton 3.6, FlashInfer 0.6.3. All kernels are warmed up for 25 iterations before timing.

Table 2 GPU specifications. Peak BF16 TFLOP/s are dense (without 2:4 structured sparsity), since the LM-head matmul is a standard dense GEMM. The ops:byte ratio contextualizes the crossover between bandwidth- and compute-limited regimes, although the exact crossover is kernel-dependent.

	H100	**H200**	**B200**	**B300**
Architecture	Hopper	Hopper	Blackwell	Blackwell
HBM capacity (GB)	80	141	192	288
HBM bandwidth (TB/s)	3.35	4.8	8.0	8.0
Peak BF16 dense (TFLOP/s)	989	989	2,250	2,250
Ops:byte ratio	295	206	281	281

Workload configurations. We test two representative LM-head shapes:

- **Small**: $D = 4{,}096$, $V = 151{,}936$ (matches Qwen3-8B and Qwen3-235B-A22B MoE),
- **Large**: $D = 8{,}192$, $V = 128{,}256$ (matches Llama 3 70B and DeepSeek-V3).

We sweep batch sizes $B \in \{1, 2, 4, 8, 16, 32, 64, 128, 256\}$ to cover the transition from memory-bound to compute-bound regimes during decoding.

Baselines.

1. **PyTorch compiled**: `torch.compile`-wrapped pipeline of matmul (dispatches to cuBLAS) → softmax → multinomial.
2. **FI1** (FlashInfer top-k/top-p): `top_k_top_p_sampling_from_logits`, the kernel used by vLLM for top-k/top-p sampling (Ye et al., 2025).
3. **FI2** (FlashInfer sampling): `sampling_from_logits`, FlashInfer's fastest sampling kernel, which also uses Gumbel-Max internally but operates on pre-materialized logits.

6.2 Standalone logits sampling

Standalone FlashSampling (Algorithms A.1-A.3) applies Gumbel-Max to pre-materialized logits. This is functionally equivalent to what FI2 (`sampling_from_logits`) already implements internally (it also uses Gumbel-Max). Thus, the FlashSampling-vs-FlashInfer comparison in Section 6.3 implicitly covers the standalone case: FI2 represents the state of the art for Gumbel-Max sampling from pre-materialized logits, and FlashSampling's advantage comes specifically from fusing the matmul with the sampling step. We focus our evaluation on the fused setting, which is the primary systems contribution.

6.3 Fused matmul and sampling

Table 3 reports FlashSampling speedups relative to all three baselines on the small config (D=4096, V=151k), which is where the largest gains are observed. All numbers are median latency over 100 timed iterations.

1. **FlashSampling is faster than all baselines at $B \leq 64$ on all GPUs.** This is the decode regime where autoregressive LLM serving operates. Across all measurements, the largest speedups are

Table 3 FlashSampling speedup vs. three baselines on the small config (D=4096, V=151k). Values > 1 indicate FlashSampling is faster; bold marks the peak per GPU within each baseline. FI1: FlashInfer top-k/top-p kernel (used in vLLM). FI2: FlashInfer Gumbel-Max kernel.

	vs. PyTorch compiled				*vs. FI1* (top-k/top-p)				*vs. FI2* (Gumbel-Max)			
B	H100	H200	B200	B300	H100	H200	B200	B300	H100	H200	B200	B300
1	1.22	1.24	1.36	1.40	**1.30**	1.29	1.37	2.06	**1.15**	**1.17**	**1.25**	**1.25**
2	1.23	1.25	1.38	1.38	1.25	**1.37**	1.46	2.05	**1.15**	**1.17**	1.24	1.23
4	1.21	1.22	1.37	1.39	**1.30**	1.35	**1.48**	1.94	1.14	1.14	1.24	1.24
8	1.22	1.25	1.41	1.35	**1.30**	**1.37**	1.47	2.24	1.13	1.14	1.22	1.23
16	1.25	1.28	1.41	1.43	1.29	1.36	**1.48**	**2.30**	1.13	1.13	1.22	1.23
32	1.29	1.30	1.39	1.39	1.27	1.32	1.36	2.10	1.11	1.08	1.14	1.14
64	1.40	**1.33**	1.36	1.38	1.29	1.23	1.23	1.90	1.10	1.01	1.03	1.04
128	**1.52**	1.26	**1.48**	**1.48**	1.25	1.04	1.17	1.72	1.03	0.84	0.96	1.00
256	1.23	1.06	1.31	1.33	1.00	0.89	1.06	1.63	0.77	0.70	0.85	0.89

1.52× vs. PyTorch compiled (H100, B=128, small) and 2.30× vs. FI1 (B300, B=16, small); within the decode regime, the peak speedup vs. PyTorch compiled is 1.43×.

2. **The advantage narrows at larger batch sizes.** On the large config, FlashSampling loses to some baselines by $B \geq 128$ because the workload moves toward a regime in which GEMM efficiency dominates. The effective-intensity model above explains this qualitative trend, but the exact crossover is implementation-dependent.

3. **Blackwell GPUs show the largest measured gains vs. FI1 in our setup.** The cause is likely a mix of architecture- and implementation-specific effects rather than raw HBM bandwidth alone.

4. **FlashSampling matches or exceeds FI2** (which also uses Gumbel-Max internally) by up to 1.25× at B=1, indicating that the main speedup comes from fusion rather than from Gumbel-Max alone.

6.4 Roofline analysis

The batch-size trend is consistent with a simple roofline view. For a BF16 baseline that materializes logits, a minimal traffic model gives

$$I_{\text{base}}(B) \approx \frac{2BVD}{2(VD + BD + BV)} = \frac{BVD}{VD + BD + BV} \quad \text{FLOP/byte},$$

where the denominator accounts for reading $\boldsymbol{W}$ and $\boldsymbol{H}$ and writing $\boldsymbol{Y}$. If sampling is fused so that the logits write/read is removed, then up to lower-order terms from the tile-winner buffer,

$$I_{\text{fused}}(B) \approx \frac{2BVD}{2(VD + BD)} = \frac{BV}{V + B} \quad \text{FLOP/byte}.$$

Table 4 FlashSampling speedup vs. three baselines on the large config (D=8192, V=128k). Values > 1 indicate FlashSampling is faster; bold marks the peak per GPU within each baseline. At $B \geq 128$ the advantage narrows and cuBLAS GEMM efficiency becomes increasingly important.

	vs. PyTorch compiled				*vs. FI1* (top-k/top-p)				*vs. FI2* (Gumbel-Max)			
B	H100	H200	B200	B300	H100	H200	B200	B300	H100	H200	B200	B300
1	1.22	1.26	**1.35**	**1.34**	1.18	**1.25**	**1.27**	1.57	**1.13**	**1.13**	**1.15**	**1.14**
2	1.22	1.23	1.31	1.33	**1.22**	1.21	1.24	1.65	**1.13**	1.12	**1.15**	**1.14**
4	1.20	1.22	1.27	1.23	1.19	1.22	1.21	1.66	1.12	1.11	1.07	1.08
8	1.21	1.22	1.24	1.26	1.20	1.22	1.19	**1.67**	1.12	1.10	1.07	1.08
16	1.21	1.22	1.26	1.28	1.20	1.22	1.21	**1.67**	1.11	1.10	1.07	1.08
32	1.21	1.23	1.25	1.29	1.18	1.20	1.20	1.64	1.10	1.09	1.06	1.07
64	**1.28**	**1.28**	1.27	1.23	1.20	1.20	1.18	1.55	1.11	1.08	1.06	1.03
128	1.13	0.93	0.97	0.96	1.01	0.81	0.86	1.09	0.90	0.72	0.77	0.76
256	0.87	0.78	0.74	0.71	0.79	0.75	0.68	0.89	0.67	0.63	0.57	0.57

Thus fusion raises the effective arithmetic intensity. In the decode regime $V \gg B$, we have $I_{\text{fused}}(B) \approx B$, so increasing B moves the workload from clearly bandwidth-bound toward a regime in which GEMM efficiency matters more.

This explains the qualitative trend that FlashSampling's advantage is largest for $B \leq 64$ and narrows around $B = 128$–256 in our measurements. The exact crossover is implementation-dependent and need not coincide with a single peak-theoretical threshold.

Importantly, the avoided logits round-trip at B=1 on the small config is only $4BV = 607{,}744$ bytes (0.607744 MB), which at 8 TB/s corresponds to 7.6×10^{-5} ms, not 0.07 ms. The observed latency gap therefore cannot be explained by raw HBM bandwidth alone.

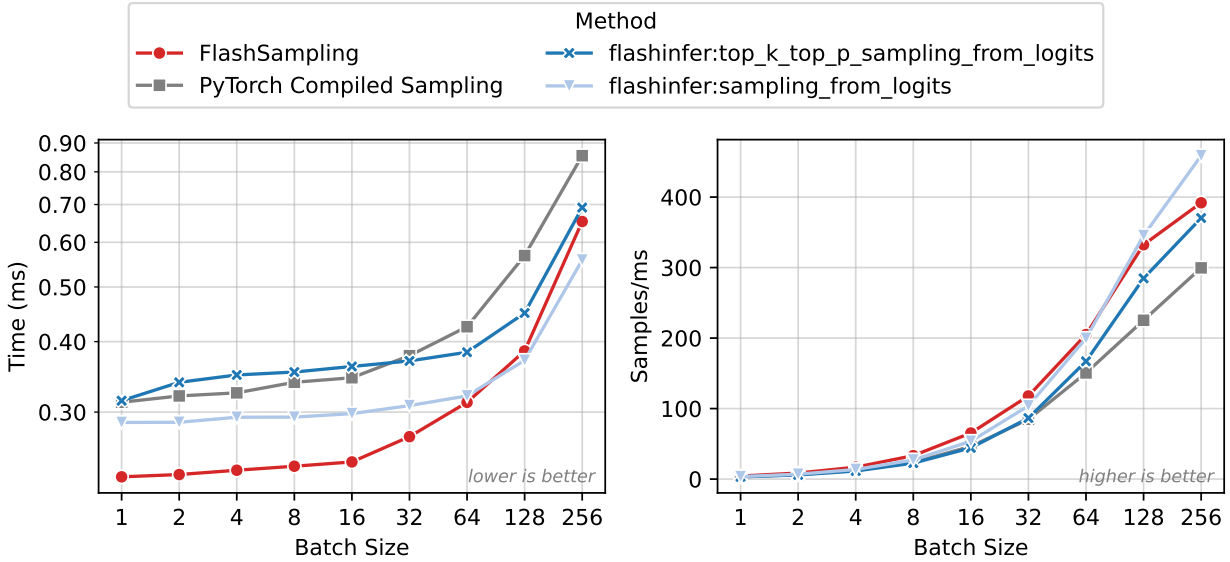

Figure 2 Absolute latency (ms) vs. batch size on B200 (small config). FlashSampling is faster than all baselines for $B \leq 64$. At $B \geq 128$ the advantage narrows as higher-GEMM-efficiency cuBLAS-based baselines become more competitive.

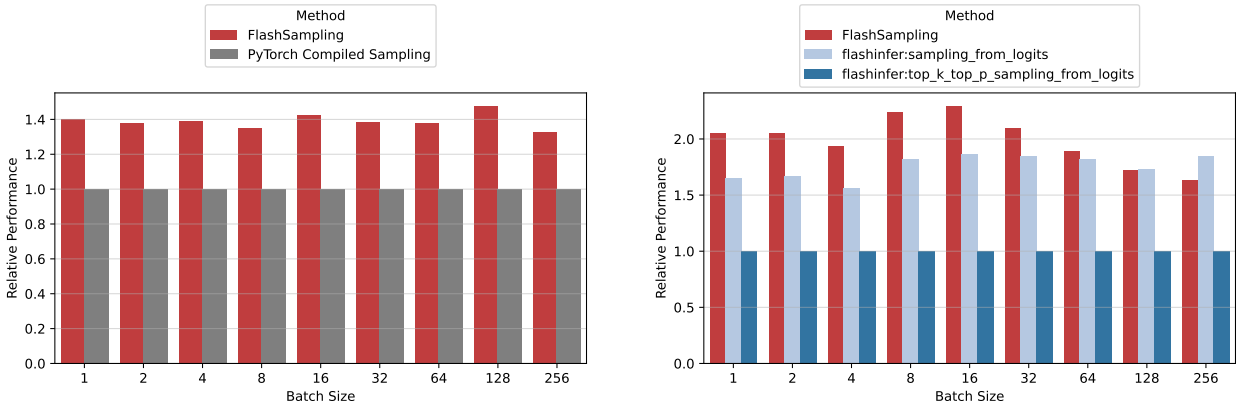

Figure 3 Relative performance on B300 (small config). Left: FlashSampling vs. PyTorch compiled (baseline = 1). Right: FlashSampling vs. FlashInfer FI1 and FI2 (baseline = 1). Against PyTorch compiled (left), FlashSampling is faster across all shown batch sizes. Against FlashInfer (right), FlashSampling is faster against FI1 throughout and against FI2 in the decode regime, with parity or a small loss at the largest batches.

6.5 End-to-end vLLM evaluation

We integrate FlashSampling into vLLM (Kwon et al., 2023) by replacing the LM-head projection and sampler with a single `fused_mm_sample_triton` call during decode. Prefill uses the standard path (logits are needed for prompt log-probabilities). Experiments run on a single B200 GPU with four models spanning a range of sizes and architectures.

For each model and concurrency level, we run 5 trials of 256 requests from the ShareGPT dataset, measuring time per output token (TPOT). We pair baseline and FlashSampling runs by trial number and report the median TPOT across trials for robustness against outlier runs.

Key observations.

1. **Largest gains on the smallest model.** Qwen3-1.7B sees 11–19% TPOT reduction because the LM head (D=2,048, V=151,936) constitutes a larger fraction of total per-token compute.

2. **Gains decrease with model size.** For Qwen3-32B (D=5,120) and gpt-oss-120b (D=2,880, V=201,088, MoE with 128 experts), attention and FFN layers dominate decode time, so gains are small overall and can be noisy; Qwen3-32B even shows a small regression at B=2.

3. **No measurable quality degradation on GSM8K.** We evaluate Qwen3-8B on GSM8K (8-shot, 500 problems): FlashSampling achieves 89.4% accuracy vs. 89.6% baseline. A bootstrap test yields $p = 0.776$; we do not detect a statistically significant difference. This is consistent with exact sampling and indicates no measurable quality loss on this benchmark.

7 Related work

Gumbel-Max and extensions. The Gumbel-Max trick for exact categorical sampling dates to Gumbel (1954) and was formalized in the machine learning context by Maddison et al. (2014). Jang et al. (2017) introduced the Gumbel-Softmax relaxation for differentiable discrete sampling, enabling gradient-based optimization through categorical variables; this is complementary to our focus on exact (non-relaxed) sampling. Huijben et al. (2022) provides a comprehensive review of Gumbel-Max extensions. Qi et al. (2020) propose methods for fast generation of large numbers of Gumbel-Max variables. FlashSampling's Group-Gumbel-Max extends this line of work with a hierarchical decomposition that enables parallel, streaming, and distributed exact sampling with formal correctness guarantees.

Kernel fusion for memory efficiency. FlashAttention (Dao et al., 2022) demonstrated that fusing attention computation to avoid materializing the $[N, N]$ attention matrix yields large speedups on GPU by reducing HBM traffic. Cut Your Losses (Wijmans et al., 2025) applies a similar principle at training time: by computing cross-entropy loss without materializing the full $[B, V]$ logits tensor, it reduces memory consumption for large-vocabulary models. FlashSampling extends this "flash" methodology from training (cross-entropy) to inference (sampling), fusing the LM-head matmul with Gumbel-Max sampling to avoid materializing logits.

LLM sampling implementations. FlashInfer (Ye et al., 2025) provides optimized GPU kernels for attention and sampling in LLM serving, including a Gumbel-Max–based sampling kernel (FI2 in

Table 5 TPOT speedup (%) on B200 with vLLM: $1 - \text{FlashSampling/baseline}$ (median $\pm$ std over 5 paired trials of 256 requests). B is the maximum number of concurrent requests. Bold marks the peak per model. FlashSampling gains decrease with model size as attention and FFN dominate decode time.

B	Qwen3-1.7B	Qwen3-8B	Qwen3-32B	gpt-oss-120b
1	$10.8 \pm 0.3\,\%$	$2.8 \pm 0.9\,\%$	$\mathbf{1.9} \pm 1.4\,\%$	$0.3 \pm 0.8\,\%$
2	$12.8 \pm 0.6\,\%$	$\mathbf{6.9} \pm 1.7\,\%$	$-1.8 \pm 0.5\,\%$	$1.7 \pm 0.5\,\%$
4	$14.5 \pm 0.4\,\%$	$3.7 \pm 0.1\,\%$	$1.6 \pm 0.0\,\%$	$\mathbf{2.4} \pm 0.3\,\%$
8	$15.2 \pm 0.4\,\%$	$3.6 \pm 0.4\,\%$	$1.1 \pm 0.1\,\%$	$2.1 \pm 0.1\,\%$
16	$17.7 \pm 9.0\,\%$	$3.0 \pm 0.4\,\%$	$1.1 \pm 0.1\,\%$	$1.8 \pm 0.4\,\%$
32	$9.7 \pm 5.1\,\%$	$4.4 \pm 1.7\,\%$	$1.3 \pm 0.3\,\%$	$1.5 \pm 2.0\,\%$
64	$\mathbf{18.7} \pm 6.8\,\%$	$5.2 \pm 2.2\,\%$	$1.2 \pm 0.3\,\%$	$1.6 \pm 0.8\,\%$

our evaluation). However, FlashInfer's sampling kernels operate on pre-materialized logits. vLLM (Kwon et al., 2023) uses FlashInfer's kernels for sampling in its decode path. FlashSampling is, to our knowledge, the first kernel to fuse the LM-head matmul with sampling, eliminating the logits round-trip to HBM entirely.

Approximate sampling methods. Rawat et al. (2019) propose sampled softmax with random Fourier features for approximate training with large vocabularies. In contrast, FlashSampling provides *exact* samples from the full categorical distribution without approximation.

Orthogonal optimizations. Speculative decoding (Leviathan et al., 2023) reduces the number of autoregressive decode steps by drafting with a smaller model and verifying with the target model. Quantization reduces per-layer compute by lowering precision. FlashSampling reduces the per-step cost of the LM-head projection and sampling, and composes naturally with both: speculative decoding still requires sampling at each verification step, and quantized models still need to produce and sample from logits.

8 Conclusion

We presented **FlashSampling**, a fused LM-head sampling design consisting of a tiled first-stage kernel and a lightweight reduction, so that the $[B, V]$ logits tensor is never materialized in HBM. Argmax decomposition over tiles guarantees that the tiled fused design is exact, while Gumbel max-stability enables grouped, streaming, and distributed variants. Profiling and the IO analysis indicate that the dominant speedup source is eliminating extra post-matmul kernels and their overhead, with sampling itself adding only a small constant overhead inside the fused stage. FlashSampling provides a practical building block for fast decoding, MoE routing, and other high-throughput stochastic operations in modern deep learning systems.

Acknowledgement

We sincerely thank Yongye Zhu, Zhuoqing Song, and Mayank Mishra for their helpful discussions and constructive feedback. We used large language models to assist in polishing the writing of this

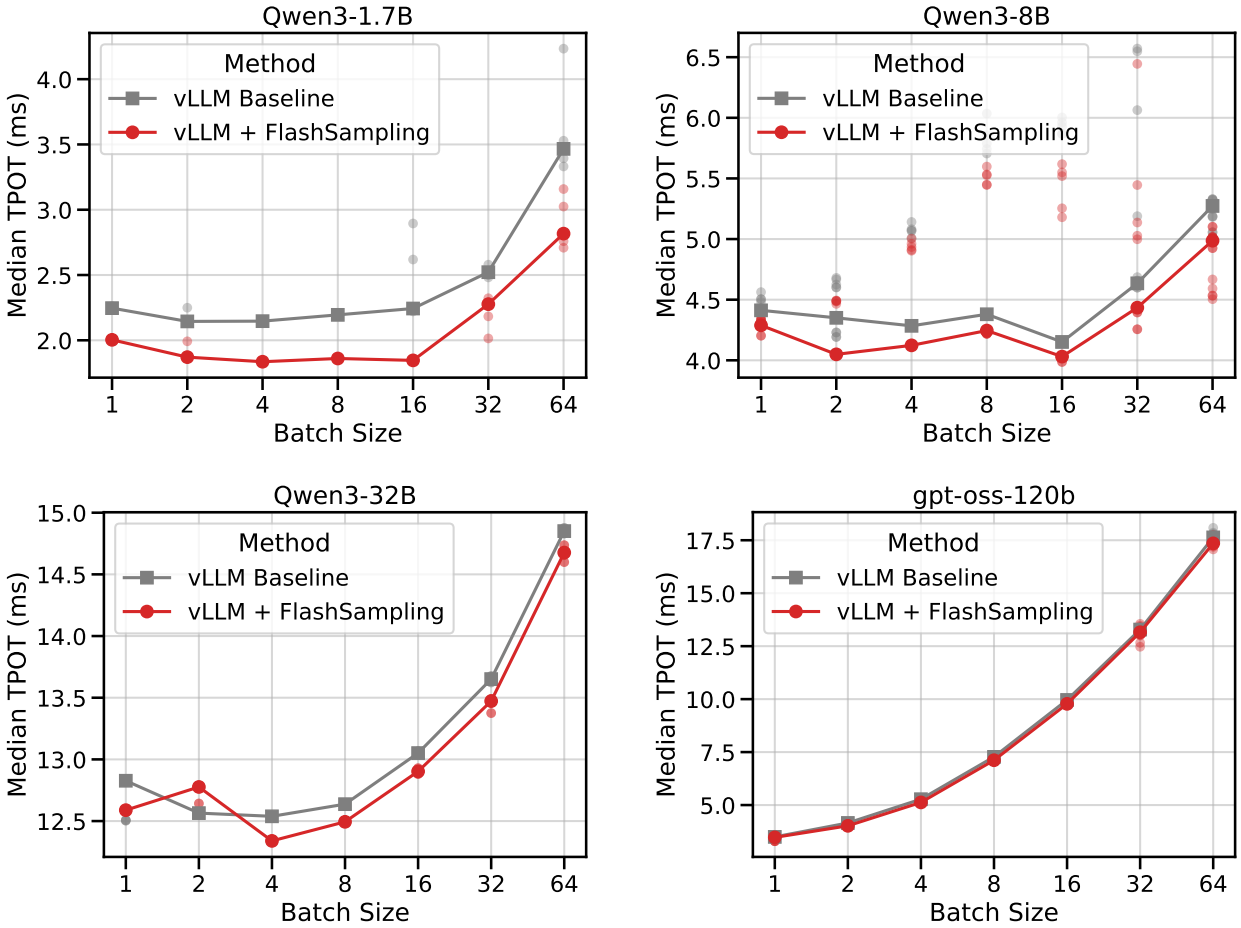

Figure 4 TPOT vs. concurrency on B200 for all four models. Top row: Qwen3-1.7B (up to 19% reduction) and Qwen3-8B (roughly 3–7%). Bottom row: Qwen3-32B and gpt-oss-120b, where gains are small and noisier because attention and FFN dominate decode time.

work.

References

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Emil Julius Gumbel. Statistical theory of extreme values and some practical applications. *National Bureau of Standards Applied Mathematics Series*, 33, 1954.

Iris AM Huijben, Wouter Kool, Max B Paulus, and Ruud JG Van Sloun. A review of the gumbel-max

trick and its extensions for discrete stochasticity in machine learning. *IEEE transactions on pattern analysis and machine intelligence*, 45(2):1353–1371, 2022.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-softmax. In *International Conference on Learning Representations*, 2017.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286, 2023.

Chris J. Maddison, Daniel Tarlow, and Tom Minka. A* sampling. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

NVIDIA. NVIDIA H100 tensor core GPU architecture. Technical report, 2022. URL https://resources.nvidia.com/en-us-data-center-overview-mc/en-us-data-center-overview/gtc22-whitepaper-hopper. Accessed: 2026-03-04.

NVIDIA. NVIDIA H100 tensor core GPU datasheet. Technical report, 2024. URL https://www.megware.com/fileadmin/user_upload/LandingPage%20NVIDIA/nvidia-h100-datasheet.pdf. Accessed: 2026-03-04.

Yiyan Qi, Pinghui Wang, Yuanming Zhang, Junzhou Zhao, Guangjian Tian, and Xiaohong Guan. Fast generating a large number of Gumbel-max variables. In *Proceedings of The Web Conference*, pages 2006–2012, 2020.

Ankit Singh Rawat, Jiecao Chen, Felix Xinnan X. Yu, Ananda Theertha Suresh, and Sanjiv Kumar. Sampled softmax with random Fourier features. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Tomas Ruiz. fmms-kernel: FMMS kernel: Fused matrix-multiplication + sampling. GitHub repository, 2026. URL https://github.com/tomasruizt/fmms-kernel. Triton implementation of fused matmul + Gumbel-Max sampling.

Erik Wijmans, Brody Koh, Roei Herzig, Jitendra Jain, Jianwei Zhu, Saurabh Kapoor, and Ross Girshick. Cut your losses in large-vocabulary language models. *arXiv preprint arXiv:2411.09009*, 2025.

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. Flashinfer: Efficient and customizable attention engine for llm inference serving. *Proceedings of Machine Learning and Systems*, 7, 2025.

Appendix

A FlashSampling Algorithm Pseudocode **22**

B Numerically Stable and Fast Gumbel Generation **23**

A FlashSampling Algorithm Pseudocode

This appendix collects detailed pseudocode for the FlashSampling variants described in the main text.

Streaming Gumbel-Max (standalone logits). Algorithm A.1 presents the basic one-pass streaming Gumbel-Max sampler over pre-materialized logits.

Algorithm A.1 Gumbel-Max sampling (standalone logits): streaming argmax over perturbed logits

Require: Logits $\boldsymbol{\ell} \in \mathbb{R}^V$, RNG state
Ensure: Sample index $i^\star \in \{1, \dots, V\}$

```
m ← −∞, i* ← 1
for i = 1 to V do
    g ← GUMBEL(0, 1)                    ▷ via g = −log(−log u), u ∼ Unif(0, 1)
    s ← ℓ_i + g
    if s > m then
        m ← s, i* ← i
    end if
end for
return i*
```

Parallel Group-Gumbel-Max. Algorithm A.2 extends streaming Gumbel-Max to a group-parallel setting where each group is processed by an independent threadblock.

Algorithm A.2 FlashSampling (parallel): Group-Gumbel-Max over groups

Require: Input $\boldsymbol{x} \in \mathbb{R}^d$, weight matrix $\boldsymbol{W} \in \mathbb{R}^{d \times V}$, group size g (so $V = mg$), RNG state
Ensure: Sample index $z \in \{1, \dots, V\}$ and optional log-normalizer $\ell_Z = \mathrm{logsumexp}(\boldsymbol{y})$

```
for k = 0 to m − 1 in parallel do
    y_k ← W_k^⊤ x ∈ R^g
    z_k ← argmax_{j∈[g]} (y_{k,j} − log(−log u_{k,j}))    ▷ u_{k,j} ∼ Unif(0, 1)
    L_k ← logsumexp(y_k)
end for
k* ← argmax_{k∈[m]} (L_k − log(−log ū_k))                ▷ ū_k ∼ Unif(0, 1)
z ← k* g + z_{k*}                   ▷ map group-local index to global vocabulary index
ℓ_Z ← logsumexp([L_0, ..., L_{m−1}])                     ▷ optional
return (z, ℓ_Z)
```

Sequential/online Group-Gumbel-Max. Algorithm A.3 provides a memory-efficient variant that streams groups one at a time.

Distributed Group-Gumbel-Max. Algorithm A.4 extends FlashSampling to tensor-parallel vocabularies sharded across multiple GPUs.

Algorithm A.3 FlashSampling (sequential/online): streaming Group-Gumbel-Max with $O(g)$ working memory

Require: Input $\boldsymbol{x} \in \mathbb{R}^d$, weight matrix $\boldsymbol{W} \in \mathbb{R}^{d \times V}$, group size g (so $V = mg$), RNG state
Ensure: Sample index $z \in \{1, \ldots, V\}$ and optional log-normalizer ℓ_Z

Initialize with the first group.
$\boldsymbol{y}_0 \leftarrow \boldsymbol{W}_0^\top \boldsymbol{x} \in \mathbb{R}^g$
$L_0 \leftarrow \mathrm{logsumexp}(\boldsymbol{y}_0)$
$z_0 \leftarrow \mathrm{argmax}_{j \in [g]} \left(y_{0,j} - \log(-\log u_{0,j})\right)$ ▷ $u_{0,j} \sim \mathrm{Unif}(0,1)$
$z \leftarrow z_0$, $\ell \leftarrow L_0$
for $k = 1$ **to** $m - 1$ **do**
  $\boldsymbol{y}_k \leftarrow \boldsymbol{W}_k^\top \boldsymbol{x} \in \mathbb{R}^g$
  $L_k \leftarrow \mathrm{logsumexp}(\boldsymbol{y}_k)$
  $\ell_{\text{new}} \leftarrow \mathrm{logsumexp}([\ell,\ L_k])$
  $p_{\text{replace}} \leftarrow \exp(L_k - \ell_{\text{new}})$ ▷ $= \frac{e^{L_k}}{e^{\ell} + e^{L_k}}$
  Draw $u \sim \mathrm{Unif}(0,1)$
  if $u < p_{\text{replace}}$ **then**
    $z_k \leftarrow \mathrm{argmax}_{j \in [g]} \left(y_{k,j} - \log(-\log u_{k,j})\right)$ ▷ sample within selected group
    $z \leftarrow kg + z_k$
  end if
  $\ell \leftarrow \ell_{\text{new}}$
end for
$\ell_Z \leftarrow \ell$ ▷ optional
return (z, ℓ_Z)

Algorithm A.4 FlashSampling (distributed, tensor-parallel vocab): communicate $O(1)$ scalars per rank

Require: World size n. Rank $k \in \{0, \ldots, n-1\}$ holds shard $\boldsymbol{W}^{(k)} \in \mathbb{R}^{d \times (V/n)}$ covering vocab indices $\{k \cdot V/n + 1, \ldots, (k+1) \cdot V/n\}$. Input $\boldsymbol{x} \in \mathbb{R}^d$, RNG state.
Ensure: Global sample index $z \in \{1, \ldots, V\}$ (and optional ℓ_Z)

On each rank k:
  compute local logits $\boldsymbol{y}^{(k)} \leftarrow (\boldsymbol{W}^{(k)})^\top \boldsymbol{x} \in \mathbb{R}^{V/n}$
  compute local log-mass $L_k \leftarrow \mathrm{logsumexp}(\boldsymbol{y}^{(k)})$
  sample local index $\widetilde{z}_k \sim \mathrm{Cat}(\mathrm{softmax}(\boldsymbol{y}^{(k)}))$ ▷ e.g., via Gumbel-Max / Group-Gumbel-Max / fused kernel
All-gather $\{(L_k, \widetilde{z}_k)\}_{k=0}^{n-1}$ to a coordinator (or perform an equivalent reduction)
Sample winning rank $k^\star \leftarrow \mathrm{argmax}_{k \in [n]} \left(L_k - \log(-\log \bar{u}_k)\right)$ ▷ $\bar{u}_k \sim \mathrm{Unif}(0,1)$
$z \leftarrow k^\star \cdot (V/n) + \widetilde{z}_{k^\star}$ ▷ convert rank-local index to global
Optionally $\ell_Z \leftarrow \mathrm{logsumexp}([L_0, \ldots, L_{n-1}])$
return z (and ℓ_Z)

B Numerically Stable and Fast Gumbel Generation

Gumbel noise can be generated as $g = -\log(-\log u)$ with $u \sim \mathrm{Unif}(0,1)$. In GPU kernels, two issues matter:

- **Numerical stability:** avoid $u = 0$ or $u = 1$ which lead to infinities.
- **Throughput:** the cost of generating random numbers and computing logs should not dominate.

Practical recipe. Given a 32-bit RNG output $r \in \{0, \ldots, 2^{32} - 1\}$, map to

$$u = \frac{r + 1}{2^{32} + 1} \in (0, 1),$$

then compute $g = -\log(-\log u)$. Many GPU RNG libraries (e.g. Philox, XORWOW) support generating floats in $(0, 1)$ directly; the above mapping is a safe fallback.

Approximate log options. If exactness in the distribution is required, the Gumbel generation must be statistically correct. However, using fast approximate log implementations can introduce small distortions. FlashSampling supports two modes:

- **Exact-math mode:** use standard log for high fidelity.
- **Fast-math mode:** use approximate logs for speed, with empirical validation that sampling bias remains negligible for target applications.

The sampling remains *algorithmically exact* with respect to the generated Gumbels; any bias comes from numeric approximations.